Тема 5. Фильтры и регулярные выражения в linux.

5.1 Фильтры в linux.

Один из основополагающих принципов Linux заключается в том, что каждый элемент должен выполнять только одну функцию и что все элементы можно с легкостью объединять друг с другом. Эти элементы — строительные блоки, которые можно собирать и таким образом строить все, что мы захотим. В этой работе мы поговорим о конкретных строительных блоках. Первый из них — фильтры.

При выполнении заданий используйте справку к командам. Также разрешается использовать поисковик, ведь даже админы наизусть не помнят всех опций. В отчете кратко напишите какую функцию выполняет каждая изученная вами команда.

Тестировать фильтры будем на следующем наборе данных:

```
Вася яблоки 20
Ксюша апельсины 5
Игорь арбузы 12
Рома груши 4
Тимофей апельсины 9
Лиза персики 7
Ксюша апельсины 5
Игорь виноград 39
Аня манго 7
Гриша ананасы 3
Коля дыни 2
Катя лаймы 14
```

С использованием вашего любимого консольного текстового редактора создайте файл и наберите в нем представленный выше набор данных. Также добавьте в файл еще 6-8 записей по вашему выбору, имена и фрукты могут повторяться. Количество фруктов от 1 до 99. В конце файла добавьте сточку «Вася яблоки 19». Запретите редактирование этого файла.

Задание 1.

1. С использованием команды head выведите первые 5 записей файла.
2. Сделайте одновременный вывод результата из предыдущего задание в терминал и в файл. Для этого используйте команду tee.

В дальнейшем используйте такой подход для выполнения остальных заданий.

3. Покажите последние 4 строки исходного файла, команда tail.
4. Выведете файл в обратном порядке командой tac.
5. Используйте команду sort и выполните сортировку исходного файла:

- по имени в прямом и обратном порядке;
- по количеству фруктов.

6. Пронумеруйте результат сортировки по имени командой nl, используйте стиль «1. Вася ...». Последовательность команд должна выглядеть так : сортировка | нумерация.
7. Найдите количество записей в исходном файле и результате из задания 5, здесь используйте команду wc. Последовательность команд должна выглядеть так : сортировка | нумерация | количество записей. Какую информацию еще может предоставить wc ?
8. Выведете только первую колонку файла, используйте команду cut.
9. Выведете первую и третью колонку файла, используйте разделитель «, ». Например «Вася, 20».
10. Используйте uniq чтобы исключить повторяющиеся строки. Помните, что повторяющиеся строки должны следовать рядом друг с другом, иначе uniq не работает. Сколько строк осталось после этой процедуры?

5.2 Регулярные выражения

Регулярные выражения - это очень мощный инструмент для поиска текста по шаблону, обработки и изменения строк, который можно применять для решения множества задач. Вот основные из них:

- Проверка ввода текста;
- Поиск и замена текста в файле;
- Пакетное переименование файлов;
- Взаимодействие с сервисами, таким как Apache;
- Проверка строки на соответствие шаблону.

Это далеко не полный список, регулярные выражения позволяют делать намного больше. Но для новых пользователей они могут показаться слишком сложными, поскольку для их формирования используется специальный язык. Но учитывая предоставляемые возможности, регулярные выражения Linux должен знать и уметь использовать каждый системный администратор.

В регулярных выражениях могут использоваться два типа символов:

Обычные символы - это буквы, цифры и знаки препинания, из которых состоят любые строки. Все тексты состоят из букв и вы можете использовать их в регулярных выражениях для поиска нужной позиции в тексте.

Метасимволы - это кое-что другое, именно они дают силу регулярным выражениям. С помощью метасимволов вы можете сделать намного больше чем поиск одного символа. Вы можете искать комбинации символов, использовать динамическое их количество и выбирать диапазоны. Все спецсимволы можно разделить на два типа, это символы замены, которые заменяют собой обычные символы, или операторы, которые указывают сколько раз может повторяться символ. Синтаксис регулярного выражения будет выглядеть таким образом:

обычный_символ спецсимвол_оператор
спецсимвол_замены спецсимвол_оператор

Если оператор не указать, то будет считаться, что символ обязательно должен встретится в строке один раз. Таких конструкций может быть много. Вот основные метасимволы, которые используют регулярные выражения bash:

\ - с обратной косой черты начинаются буквенные спецсимволы, а также он используется если нужно использовать спецсимвол в виде какого-либо знака препинания;
^ - указывает на начало строки;
$ - указывает на конец строки;
* - указывает, что предыдущий символ может повторяться 0 или больше раз;
+ - указывает, что предыдущий символ должен повторится больше один или больше раз;
? - предыдущий символ может встречаться ноль или один раз;
{n} - указывает сколько раз (n) нужно повторить предыдущий символ;
{N,n} - предыдущий символ может повторяться от N до n раз;
. - любой символ кроме перевода строки;
[az] - любой символ, указанный в скобках;
x|y - символ x или символ y;
[^az] - любой символ, кроме тех, что указаны в скобках;
[a-z] - любой символ из указанного диапазона;
[^a-z] - любой символ, которого нет в диапазоне;
\b - обозначает границу слова с пробелом;
\B - обозначает что символ должен быть внутри слова, например, ux совпадет с uxb или tuxedo, но не совпадет с Linux;
\d - означает, что символ - цифра;
\D - нецифровой символ;
\n - символ перевода строки;

\s - один из символов пробела, пробел, табуляция и так далее;
\S - любой символ кроме пробела;
\t - символ табуляции;
\v - символ вертикальной табуляции;
\w - любой буквенный символ, включая подчеркивание;
\W - любой буквенный символ, кроме подчеркивания;
\uXXX - символ Unicdoe.
Важно отметить, что перед буквенными спецсимволами нужно использовать косую черту, чтобы указать, что дальше идет спецсимвол. Правильно и обратное, если вы хотите использовать спецсимвол, который применяется без косой черты в качестве обычного символа, то вам придется добавить косую черту.

Несколько примеров использования регулярных выражений
Два очень полезные спецсимвола - это ^ и $, которые обозначают начало и конец строки. Например, мы хотим получить всех пользователей, зарегистрированных в нашей системе, имя которых начинается на s. Тогда можно применить регулярное выражение "^s". Вы можете использовать команду egrep:
```
egrep "^s" /etc/passwd
```
Если мы хотим отбирать строки по последнему символу в строке, что для этого можно использовать $. Например, выберем всех системных пользователей, без оболочки, записи о таких пользователях заканчиваются на false:
```
egrep "false$" /etc/passwd
```
Чтобы вывести имена пользователей, которые начинаются на s или d используйте такое выражение:
```
egrep "^[sd]" /etc/passwd
```
Такой же результат можно получить, использовав символ "|". Первый вариант более пригоден для диапазонов, а второй чаще применяется для обычных или/или:
```
egrep "^[s|d]" /etc/passwd
```
Теперь давайте выберем всех пользователей, длина имени которых составляет не три символа. Имя пользователя завершается двоеточием. Мы можем сказать, что оно может содержать любой буквенный символ, который должен быть повторен три раза, перед двоеточием:
```
egrep "^\w{3}:" /etc/passwd
```
egrep — это далеко не единственная программа, использующая регулярные выражения. Среди других программ, в которых применяются регулярные выражения, можно отметить awk, sed, perl, разные редакторы, вроде Vim и emacs.

Экранирование символов: Для поиска спецсимволов, например точки или пробела, можно воспользоваться экранированием символов с использованием обратной косой черты.
```
egrep "\ " /etc/passwd
```
В случае если требуется выполнить поиск по сложному шаблону можно использовать классы символов. Например id пользователя в диапазоне 100-199:
```
egrep ":1[0-9][0-9]:" /etc/passwd
```
В класс можно включить также и буквы, например [XYZ], [aA] или [a-zA-Z]. Если нужно найти тире (-), то этот знак должен идти в начале группы. А вот если нужно найти символ ^ — его не надо ставить в начало группы. В результате, например, если нас интересуют все цифры, а так же символы - и ^, то соответствующее регулярное выражение будет выглядеть так:
```
egrep [-0-9^]
```
Поиск пользователя первая буква которого в диапазоне [r-s], вторая в [s-z], третья в [a-n].
```
egrep "^[r-s][s-z][a-n]" /etc/passwd
```
Более подробно на русском https://habr.com/ru/company/ruvds/blog/533334/

Задание 2. работаем с командой egrep.
1. Из исходного файла выберите только те строки которые содержат апельсины, выведите их количество.
2. Выберите только те строки которые содержат Коля или Ксюша, выведите их количество.
3. Выберите только те строки которые содержат персики или Игорь, выведите их количество.
4. Выберите только те строки которые содержат ананасы и Гриша, выведите их количество.
5. Выберите только те строки где 19 яблок, выведите их количество.
6. Выберите только те строки где фрукты начинаются на а или я, выведите их количество.
7. Выберите только те строки где Имена 4 буквы, выполните сортировку по имени.
8. Выберите только те строки где количество фруктов более 19, выполните сортировку по количеству фруктов.
9. Выберите только те строки которые **не** содержат яблоки и виноград.

5.3 Потоковый текстовый редактор sed
sed – это потоковый редактор текста (от stream editor), с помощью которого можно выполнять с файлами множество операций вроде поиска и замены, вставки или удаления. При этом чаще всего он используется именно для поиска и замены.
sed позволяет редактировать файлы, не открывая, что существенно ускоряет работу, чем при использовании того же vi. Помимо этого, SED поддерживает регулярные выражения, с помощью которых можно выполнять сложное сопоставление шаблонов.
Редактор sed обладает очень серьезным функционалом, однако в рамках этого курса мы рассмотрим только несколько примеров использования этого инструмента. Описание работы sed на русском языке и еще больше примеров вы сможете найти в сети интернет.
Общий синтаксис команды sed выглядит так:
sed опции -e команды файл
Основные опции:
-n, --quiet - не выводить содержимое буфера шаблона в конце каждой итерации;
-e - команды, которые надо выполнить для редактирования;
-f - прочитать команды редактирования из файла;
-i - сделать резервную копию файла перед редактированием;
-l - указать свою длину строки;
-r - включить поддержку расширенного синтаксиса регулярных выражений;
-s - если передано несколько файлов, рассматривать их как отдельные потоки, а не как один длинный.

КАК РАБОТАЕТ SED. Теперь нужно понять как работает команда sed. У утилиты есть два буфера, это активный буфер шаблона и дополнительный буфер. Оба изначально пусты. Программа выполняет заданные условия для каждой строки в переданном ей файле.
sed читает одну строку, удаляет из неё все завершающие символы и символы новой строки и помещает её в буфер шаблона. Затем выполняются переданные в параметрах команды, с каждой командой может быть связан адрес, это своего рода условие и команда выполняется только если подходит условие.
Когда всё команды будут выполнены и не указана опция -n, содержимое буфера шаблона выводится в стандартный поток вывода перед этим добавляется обратно символ перевода строки. если он был удален. Затем запускается новая итерация цикла для следующей строки.
Если не используются специальные команды, например, D, то после завершения одной итерации цикла содержимое буфера шаблона удаляется. Однако содержимое предыдущей строки хранится в дополнительном буфере и его можно использовать.

АДРЕСА SED. Каждой команде можно передать адрес, который будет указывать на строки, для которых она будет выполнена:

номер - позволяет указать номер строки, в которой надо выполнять команду;
первая~шаг - команда будет выполняется для указанной в первой части сроки, а затем для всех с указанным шагом;
$ - последняя строка в файле;
/регулярное_выражение/ - любая строка, которая подходит по регулярному выражению. Модификатор l указывает, что регулярное выражение должно быть не чувствительным к регистру;
номер, номер - начиная от строки из первой части и заканчивая строкой из второй части;
номер, /регулярное_выражение/ - начиная от сроки из первой части и до сроки, которая будет соответствовать регулярному выражению;
номер, +количество - начиная от номера строки указанного в первой части и еще плюс количество строк после него;
номер, ~число - начиная от строки номер и до строки номер которой будет кратный числу.
Если для команды не был задан адрес, то она будет выполнена для всех строк. Если передан один адрес, команда будет выполнена только для строки по этому адресу. Также можно передать диапазон адресов. Тогда адреса разделяются запятой и команда будет выполнена для всех адресов диапазона.

СИНТАКСИС РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ. Вы можете использовать такие же регулярные выражения, как и для Bash и популярных языков программирования. Вот основные операторы, которые поддерживают регулярные выражения sed Linux:
* - любой символ, любое количество;
\+ - как звездочка, только один символ или больше;
\? - нет или один символ;
\{i\} - любой символ в количестве i;
\{i,j\} - любой символ в количестве от i до j;
\{i,\} - любой символ в количестве от i и больше.

КОМАНДЫ SED. Если вы хотите пользоваться sed, вам нужно знать команды редактирования. Рассмотрим самые часто применяемые из них:
- комментарий, не выполняется;
q - завершает работу сценария;
d - удаляет буфер шаблона и запускает следующую итерацию цикла;
p - вывести содержимое буфера шаблона;
n - вывести содержимое буфера шаблона и прочитать в него следующую строку;
s/что_заменять/на_что_заменять/опции - замена символов, поддерживаются регулярные выражения;
y/символы/символы - позволяет заменить символы из первой части на соответствующие символы из второй части;
w - записать содержимое буфера шаблона в файл;
N - добавить перевод строки к буферу шаблона;
D - если буфер шаблона не содержит новую строку, удалить его содержимое и начать новую итерацию цикла, иначе удалить содержимое буфера до символа перевода строки и начать новую итерацию цикла с тем, что останется;
g - заменить содержимое буфера шаблона, содержимым дополнительного буфера;
G - добавить новую строку к содержимому буфера шаблона, затем добавить туда же содержимое дополнительного буфера.
Утилите можно передать несколько команд, для этого их надо разделить точкой с запятой или использовать две опции -e. Теперь вы знаете всё необходимое и можно переходить к примерам.

ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ SED

Теперь рассмотрим примеры sed Linux, чтобы у вас сложилась целостная картина об этой утилите. Давайте сначала выведем из файла строки с пятой по десятую. Для этого воспользуемся командой -p. Мы используем опцию -n чтобы не выводить содержимое буфера шаблона на каждой итерации, а выводим только то, что нам надо. Если команда одна, то опцию -e можно опустить и писать без неё:

```
sed -n '5,10p' /etc/group
```

Или можно вывести весь файл, кроме строк с первой по двадцатую. Здесь наоборот, опцию -n не указываем, чтобы выводилось всё, а с помощью команды d очищаем ненужное.

```
sed '1,20d' /etc/group
```

Дальше рассмотрим замену в sed. Это самая частая функция, которая применяется вместе с этой утилитой. Заменим вхождения слова root на losst в том же файле и выведем всё в стандартный вывод:

```
sed 's/root/losst/g' /etc/group
```

Флаг g заменяет все вхождения, также можно использовать флаг i, чтобы сделать регулярное выражение sed не зависимым от регистра. Для команд можно задавать адреса. Например, давайте выполним замену 0 на 1000, но только в строках с первой по десятую:

```
sed '1,10 s/0/1000/g' /etc/group
```

Переходим ещё ближе к регулярным выражениям, удалим все пустые строки или строки с комментариями из конфига сети:

```
sed '/^#\|^$/d'  /etc/network/interfaces
```

Регулярные выражения можно использовать и при замене. Например, заменим все вхождения p в начале строки на losst_p:

```
sed 's/[$p*]/losst_p/g' /etc/group
```

Задание 3. работаем с командой sed. Будет сложно и непонятно. Если совсем не понятно как решить задачу ищем подсказки в интернете.

1. Выведите первые 5 записей файла.
2. Выведите строки с 5 по 10.
3. Выведите все кроме строк с 5 по 10.
4. Заменить яблоки на лимоны.
5. Заменить яблоки на лимоны, а манго на хурма одной командой.
6. Добавить в конце каждого числа «штук», чтобы получилось «Вася яблоки 20 штук».
6. Добавить в начале каждой записи «У», склонение у имени, и знаки препинания чтобы получилось «У Васи яблоки, 20 штук.» .
7. Готовим данные к импорту в базу данных в виде таблицы. Разделитель между полями «|», первая строка полученного файла должна содержать заголовок таблицы «ID|name|fruit|amount». Можете использовать здесь команды cat, echo и nl. Должно получиться так:

```
ID|name|fruit|amount
1|Вася|яблоки|20
2|Ксюша|апельсины|5
```